

POLITECNICO
MILANO 1863

Ingegneria del Software

Ingegneria Informatica – Prof. San Pietro

Esercitazione:

Abstract Data Type

My Contacts:
Giuseppe Sorrentino
giuseppe.sorrentino@polimi.it

TdE 14-09-2019

- Si consideri la seguente specifica della classe PriorityQueue che modella una coda con priorità di elementi (non nulli).

```
class PriorityQueue<T> {
   // Aggiunge in coda l'elemento element con priorità priority.
   // Solleva una DuplicateException se l'elemento è già presente nella coda.
   public void push(T element, int priority) throws DuplicateException;

   // Ritorna l'elemento più vecchio tra quelli a priorità massima presenti
   // nella coda, senza rimuoverlo dalla coda.
   // Solleva una EmptyException se la coda è vuota.
   public T /* pure */ peek() throws EmptyException;

   // Ritorna e rimuove l'elemento più vecchio tra quelli a priorità massima
   // presenti nella coda.
   // Solleva una EmptyException se la coda è vuota.
   public T pull() throws EmptyException;

   // Ritorna il numero totale di elementi presenti nella coda.
   public int /* pure */ size();

   // Ritorna il numero di elementi aventi priorità priority presenti nella coda.
   public int /* pure */ size(int priority);

   // Ritorna true se e solo se l'element elem è presente in coda.
   public boolean /* pure */ contains(T element);

   // Ritorna la priorità di element, se element è presente in coda.
   // Solleva una UnknownElement altrimenti.
   public int /* pure */ getPriority(T element);
```

TdE 14-09-2019: Quesito b - Soluzione

Si assuma ora che la classe `PriorityQueue` non includa il metodo `iterator` ma includa invece il metodo `get` definito come segue.

```
    // Ritorna l'elemento i-esimo presente nella coda secondo l'ordine previsto
    // (gli elementi sono ordinati per priorità decrescente e, a pari priorità, dal meno recente
    // al più recente).
    // @Override
    // public T /* pure */ get(int i);
}
```

Per tale versione della classe si fornisca l'invariante pubblico.

TdE 14-09-2019: Quesito b

Partiamo da ciò che sicuramente è vero.
Abbiamo un metodo size() ed un size(priority). Il primo mi restituisce la size della coda, il secondo la size degli elementi con una certa priority.

TdE 14-09-2019: Quesito b

Partiamo da ciò che sicuramente è vero.
Abbiamo un metodo size() ed un size(priority). Il primo mi restituisce la size della coda, il secondo la size degli elementi con una certa priority.

1. Ciò che sicuramente vero, è che, se sommo il numero di elementi **per ogni priorità** otterrò lo stesso numero di elementi della size(), quindi:

TdE 14-09-2019: Quesito b

Partiamo da ciò che sicuramente è vero.
Abbiamo un metodo size() ed un size(priority). Il primo mi restituisce la size della coda, il secondo la size degli elementi con una certa priority.

1. Ciò che sicuramente vero, è che, se sommo il numero di elementi **per ogni priorità** otterrò lo stesso numero di elementi della size(), quindi:

- //@public invariant

\sum (int p; ; size(p)) == size() &&

TdE 14-09-2019: Quesito b

Partiamo da ciò che sicuramente è vero.

TdE 14-09-2019: Quesito b

Partiamo da ciò che sicuramente è vero.

2. Essendo una coda di priorità, è anche vero che, per ogni elemento diverso dal primo, tutti avranno priorità minore o uguale. Ovvero, la coda ha priorità decrescente, ed in 0 avrà l'elemento a priorità massima

TdE 14-09-2019: Quesito b

Partiamo da ciò che sicuramente è vero.

2. Essendo una coda di priorità, è anche vero che, per ogni elemento diverso dal primo, tutti avranno priorità minore o uguale. Ovvero, la coda ha priorità decrescente, ed in 0 avrà l'elemento a priorità massima

- \forall(int i; 1 <= i < size(); getPriority(get(i-1)) >= getPriority(get(i)))

TdE 14-09-2019: Quesito b

Partiamo da ciò che sicuramente è vero.

TdE 14-09-2019: Quesito b

Partiamo da ciò che sicuramente è vero.

3. Gli elementi della coda sono oggetti, devo controllare non siano null

TdE 14-09-2019: Quesito b

Partiamo da ciò che sicuramente è vero.

3. Gli elementi della coda sono oggetti, devo controllare non siano null

- //@ \forall(int i; 0 <= i < size(); get(i) != null) &&

TdE 14-09-2019: Quesito b

Partiamo da ciò che sicuramente è vero.

TdE 14-09-2019: Quesito b

Partiamo da ciò che sicuramente è vero.

4. Dato che è una coda, deve valere l'operazione contains(), ovvero un elemento **e** fa parte della coda se esiste un indice **i** tale che a quell'indice, l'elemento nella coda è proprio **e**

TdE 14-09-2019: Quesito b

Partiamo da ciò che sicuramente è vero.

4. Dato che è una coda, deve valere l'operazione contains(), ovvero un elemento **e** fa parte della coda se esiste un indice **i** tale che a quell'indice, l'elemento nella coda è proprio **e**

```
\forall(T el; ; contains(el) <==> //@ \exists(int i; 0 <= i < size();
                                                get(i).equals(el)))
```

TdE 14-09-2019: Quesito b

Partiamo da ciò che sicuramente è vero.

TdE 14-09-2019: Quesito b

Partiamo da ciò che sicuramente è vero.

5. Un altro controllo che posso fare è che, essendo una coda, l'elemento in indice 0 debba corrispondere al risultato di peek(). Stando anche all'ordine della nostra coda, sarà l'elemento a priorità maggiore

TdE 14-09-2019: Quesito b

Partiamo da ciò che sicuramente è vero.

5. Un altro controllo che posso fare è che, essendo una coda, l'elemento in indice 0 debba corrispondere al risultato di peek(). Stando anche all'ordine della nostra coda, sarà l'elemento a priorità maggiore

\get(0).equals(peek())

TdE 14-09-2019: Quesito b

Partiamo da ciò che sicuramente è vero.

TdE 14-09-2019: Quesito b

Partiamo da ciò che sicuramente è vero.

6. Ancora più nel dettaglio, abbiamo usato un metodo size(priority), quindi andiamo a mettere come condizione che l'output di size(priority) sia effettivamente il numero di elementi di priorità «priority»

TdE 14-09-2019: Quesito b

Partiamo da ciò che sicuramente è vero.

6. Ancora più nel dettaglio, abbiamo usato un metodo size(priority), quindi andiamo a mettere come condizione che l'output di size(priority) sia effettivamente il numero di elementi di priorità «priority»

- \forall(int p; ; \numof(T el; contains(el); getPriority(el) == p) == size(p))

TdE 14-09-2019: Quesito b

Partiamo da ciò che sicuramente è vero.

TdE 14-09-2019: Quesito b

Partiamo da ciò che sicuramente è vero.

7. In ultimo, controlliamo che non ci siano duplicati

TdE 14-09-2019: Quesito b

Partiamo da ciò che sicuramente è vero.

7. In ultimo, controlliamo che non ci siano duplicati

- \forall(int i; 0 <= i < size(); \forall(int j; 0 <= j < size() && i != j; !get(i).equals(get(j))))

TdE 14-09-2019: Quesito b - Soluzione

Soluzione

```
//@public invariant
//@ \sum (int p; ; size(p)) == size() &&
//@ \forall(int i; 1 <= i < size(); getPriority(get(i-1)) >= getPriority(get(i))) &&
//@ \forall(int i; 0 <= i < size(); get(i) != null) &&
//@ \forall(T el; ; contains(el) <==>
//@    \exists(int i; 0 <= i < size(); get(i).equals(el))) &&
//@ get(0).equals(peek()) &&
//@ \forall(int p; ;
//@    \numof(T el; contains(el); getPriority(el) == p) == size(p))
//@ \forall(int i; 0 <= i < size();
//@    \forall(int j; 0 <= j < size() && i != j; !get(i).equals(get(j)) ) )
```

TdE 26/06/2019

Si consideri la seguente specifica della classe RecentContacts che modella la lista dei contatti di una chat ordinati sulla base dell'ultima interazione (dalla più alla meno recente).

```
public class RecentContacts {
  // Crea una lista di contatti vuota.
  public RecentContacts();

  // Aggiunge un nuovo contatto in fondo alla lista.
  // Lancia l'eccezione ExistingContactException
  // se il contatto è già in lista.
  public void add(String contact) throws ExistingContactException;

  // Rimuove il contatto dalla lista.
  // Lancia l'eccezione UnknownContactException se il contatto non è in lista.
  public void remove(String contact) throws UnknownContactException;

  // Metodo invocato quando avviene un'interazione
  // (invio o ricezione di un messaggio) con il contatto contact.
  // Lancia l'eccezione UnknownContactException se il contatto non è in lista.
  public void notifyInteraction(String contact) throws UnknownContactException;

  // Restituisce il contatto in posizione pos.
  public /*@ pure @*/ String get(int pos);

  // Restituisce il numero di contatti presenti in lista.
  public /*@ pure @*/ int size();
}
```

TdE 26/06/2019: quesito b

- Si specifichi in JML l'invariante pubblico del tipo RecentContacts. Esistono propriet`a rilevanti per la classe che non si possono esprimere con un invariante?

TdE 26/06/2019: quesito b

- Si specifichi in JML l'invariante pubblico del tipo RecentContacts. Esistono propriet`a rilevanti per la classe che non si possono esprimere con un invariante?

- Non è possibile esprimere proprietà evolutive, in particolare non si può affermare che l'ordine con cui si trovano i contatti riflette l'ordine di invocazione dei metodi add, remove e notifyInteraction. Tale proprietà viene comunque garantita dalle post-condizioni dei modificatori.

TdE 26/06/2019: quesito b - soluzione

Soluzione

```
//@public invariant
//@ size() >=0 &&
//@ \forall(int i; 0<=i<size(); get(i)!=null) &&
//@ \forall(int i; 0<=i<size();
//@   \forall(int j; 0<=j<size() && i!=j; !get(i).equals(get(j))))
```

La size **non può** essere **negativa**

Gli elementi **non** devono essere null

Gli elementi devono essere diversi tra loro

TdE 26/06/2019: quesito b - soluzione

Soluzione

```
//@public invariant
//@ size() >=0 &&
//@ \forall(int i; 0<=i<size(); get(i)!=null) &&
//@ \forall(int i; 0<=i<size();
//@   \forall(int j; 0<=j<size() && i!=j; !get(i).equals(get(j))))
```

TdE 16/01/2018

- Data la seguente classe:

```
public class Bunch<T> {
// OVERVIEW: Una collezione di oggetti di tipo T.
// Gli elementi sono disposti in ordine qualunque e sono  di molteplicità qualunque.
// Il valore null non  fa parte della collezione
// Usa equals per confrontare gli oggetti

//@ensures (*costruisce il bunch vuoto*)
public Bunch();

// Aggiunge x a this
// Lancia eccezione se x e' null
public void ins(T x) throws NullPointerException;

// Elimina una comparsa di x
// Restituisce il numero di elementi rimanenti uguali a x.
// Lancia eccezione se x e' null
public int del(T x)  throws NullPointerException;

// Restituisce il numero di elementi uguali a x
public /*@ pure @*/ int num(T x)
```

TdE 16/01/2018 – quesito a

- Scrivere la specifica JML del metodo del(T x) e un invariante pubblico per la classe.

TdE 16/01/2018 – quesito a - soluzione

- Scrivere la specifica JML del metodo del(T x) e un invariante pubblico per la classe.

```
\\@Public Invariant
\forall(T t;;num(t) > = 0);
```

TdE 16/01/2018 – quesito a - soluzione

- Scrivere la specifica JML del metodo del(T x) e un invariante pubblico per la classe.

```
//@signals
public int del(T x) throws NullPointerException
```

TdE 16/01/2018 – quesito a - soluzione

- Scrivere la specifica JML del metodo del(T x) e un invariante pubblico per la classe.

```
//@signals(NullPointerException e) x == null;
public int del(T x) throws NullPointerException
```

TdE 16/01/2018 – quesito a - soluzione

- Scrivere la specifica JML del metodo del(T x) e un invariante pubblico per la classe.

```
//@ensures

//@signals(NullPointerException e) x == null;
public int del(T x) throws NullPointerException
```

TdE 16/01/2018 – quesito a - soluzione

- Scrivere la specifica JML del metodo del(T x) e un invariante pubblico per la classe.

```
//@ensures
        x!=null &&
```

```
//@signals(NullPointerException e) x == null;
public int del(T x) throws NullPointerException
```

TdE 16/01/2018 – quesito a - soluzione

- Scrivere la specifica JML del metodo del(T x) e un invariante pubblico per la classe.

```
//@ensures
        x!=null &&
        (\forall T t;!t.equals(x); num(t) == \old(num(t)) &&

//@signals(NullPointerException e) x == null;
public int del(T x) throws NullPointerException
```

TdE 16/01/2018 – quesito a - soluzione

- Scrivere la specifica JML del metodo del(T x) e un invariante pubblico per la classe.

```
//@ensures
        x!=null &&
        (\forall T t;!t.equals(x); num(t) == \old(num(t)) &&
                \old(num(x)==0) ? num(x) ==0 && \result==0 :

//@signals(NullPointerException e) x == null;
public int del(T x) throws NullPointerException
```

TdE 16/01/2018 – quesito a - soluzione

- Scrivere la specifica JML del metodo del(T x) e un invariante pubblico per la classe.

```
//@ensures
        x!=null &&
        (\forall T t;!t.equals(x); num(t) == \old(num(t)) &&
                \old(num(x)==0) ? num(x) ==0 && \result==0 :
                        num(x) ==\old(num(x)-1) && \result==num(x);
//@signals(NullPointerException e) x == null;
public int del(T x) throws NullPointerException
```

TdE 16/01/2018 – quesito b

- Completare, con AF e RI (esprimibili con opportuni inviarianti) e con il codice del metodo del(T x), la seguente implementazione della classe Bunch. Il rep `e costituito da una lista di coppie, ciascuna contenente un elemento, se presente, e la sua molteplicit`a. I dettagli possono essere ricavati dall’esame del codice.

TdE 16/01/2018 – quesito b

```
public class Bunch<T> {
//rep:
     private List<Pair> elem;
     private static class Pair {
          // OVERVIEW: Tipo immutabile rappresentante una coppia (T,int)
          T t;
          int i;
          Pair(T y, int j) {
               t = y;
               i = j;
          }

          boolean contains(T x) {
               return (x.equals(t));
          }
     }
```

```
public Bunch(){
     elem = new ArrayList<Pair>();
}
private int find(T x) {
     for (int j=0; j<elem.size(); j++;) {
          Pair p = elem.get(j);
          if (p.contains(x)) return j;
     }
     return -1;
}
public /*@ pure @*/ int num(T x) {
     int j = find(x);
     if (j==-1) return 0;
     return elem.get(j).i;
}
public void ins(T x) throws NullPointerException {
     if (x==null) throw new NullPointerException();
     i = find(x);
     if (i>-1) elem.set(i,new Pair(x,1+elem.get(i).i));
     else elem.add(new Pair(x,1));
}
public int del(T x) throws NullPointerException {
     if (x==null) throw new NullPointerException();
     int j = find(x);
     if (j==-1) return 0;

     }
 }
```

TdE 16/01/2018 – quesito b - soluzione

- Per prima cosa, implementiamo il metodo:

TdE 16/01/2018 – quesito b - soluzione

- Per prima cosa, implementiamo il metodo:

```
public int del(T x) throws NullPointerException {
  if (x==null) throw new NullPointerException();
  int j = find(x);
  if (j==-1) return 0;
  int temp=elem.get(j).i-1; // tolgo 1, al numero di comparse di x
  if (temp==0) {  // se il valore scende a 0, lo rimuovo
    elem.remove(j);
  } else { // altrimenti aggiorno
    elem.set(j,new Pair(x,temp));
  }
  return temp;
}
```

TdE 16/01/2018 – quesito b - soluzione

- Ora passiamo al RI. Il representation invariant mi dice un qualcosa che è «sempre vero» negli attributi della classe. In qusto caso, la lista è diversa da null e il valore di occorrenze, se l'elemento è inserito nella lista, è sempre maggiore di 0

Soluzione

```
private invariant elem != null &&
(\forall int j; 0<=j && j<elem.size();
 elem.get(j).i>0);
```

TdE 16/01/2018 – quesito b - soluzione

- AF, invece, è quella funzione che mi permette di esprimere i metodi pure. In questo caso, dobbiamo descrivere l'output di num()

Soluzione

```
private invariant
(\forall int j; 0<=j && j<elem.size();
    num(elem.get(j).t)==elem.get(j).i) &&
(\forall T x;;
    !(\exists int j; 0<=j && j<elem.size(); x.equals(elem.get(j).t)
    ==> num(x)==0;
```

TdE 25/01/2019

Si consideri il seguente tipo di dato astratto che definisce una `MultiList<T>`. Una `MultiList<T>` è un atomo (ovvero un elemento di un generico tipo `T`) oppure una lista di `MultiList<T>`.
Per esempio, `(1 (2) 3 4 ((5 6) 7))` è una rappresentazione testuale di una `MultiList<Integer>`, il cui primo elemento (*first*) è l'atomo `1` e il cui resto (*rest*) è la `MultiList ((2) 3 4 ((5 6) 7))`, ovvero la `MultiList` privata del suo primo elemento.
Altri esempi di `MultiList<Integer>` sono l'atomo `3` e la `MultiList (3)`. Si noti che queste due `MultiList<Integer>` sono diverse: la prima è una `MultiList` "atomica", mentre la seconda è una "vera" `MultiList` che contiene al suo interno l'atomo `3`.

TdE 25/01/2019 – quesito b

- Si consideri ora la seguente implementazione

```
public class MultiList<T> {
  private T atom;
  private List<MultiList<T>> list;

  public MultiList() {
    atom = null; list = new ArrayList<MultiList<T>>();
  }
  public MultiList(T x) { // richiede che x non sia null
    atom = x; list = null;
  }
  public /*@ pure @*/ boolean isEmpty() {
    return atom==null && list.isEmpty();
  }
  public /*@ pure @*/ boolean isAtom() {
    return atom != null;
  }
  public /*@ pure @*/ T getAtom() throws EmptyException, MalformedListException {
    if(isEmpty()) throw new EmptyException();
    if(!isAtom()) throw new MalformedListException();
    return atom;
  }
  public /*@ pure @*/ MultiList<T> first() throws EmptyException, MalformedListException {
    if(isEmpty()) throw new EmptyException();
    if(isAtom()) throw new MalformedListException();
    return list.get(0);
  }
  public /*@ pure @*/ MultiList<T> rest() throws EmptyException, MalformedListException {
    if(isEmpty()) throw new EmptyException();
    if(isAtom()) throw new MalformedListException();
    if(list.size()==1) return null;
    MultiList<T> res = new MultiList<T>();
    res.list.addAll(list.sublist(1,list.size()));
    return res;
  }
```

```
  public void addFirst(MultiList<T> x) throws MalformedListException {
    if(isAtom()) throw new MalformedListException();
    list.add(0, x);
  }
  public void removeFirst() throws EmptyException, MalformedListException {
    if(isEmpty()) throw new EmptyException();
    if(isAtom()) throw new MalformedListException();
    list.remove(0);
  }
  public /*@ pure @*/ boolean contains(MultiList<T> x) MalformedListException {
    if(isAtom()) throw new MalformedListException();
    if(isEmpty()) return false;
    if(first().equals(x)) return true;
    if(!first().isAtom() && first().contains(x)) return true;
    return rest()!=null && rest().contains(x);
  }
  @Override
  public /*@ pure @*/ boolean equals(Object that) {
    // .....
  }
  public /*@ pure @*/ MultiList<T> copy() {
    // .....
  }
}
```

TdE 25/01/2019 – quesito b

- Scrivere in JML l'invariante di rappresentazione (Rep Invariant) per MultiList<T>.

TdE 25/01/2019 – quesito b

- Scrivere in JML l'invariante di rappresentazione (Rep Invariant) per MultiList<T>.

Soluzione

```
//@private invariant ((atom!=null && list==null) ||
//@                    (atom==null && list!=null)) &&
//@                   (list!=null ==>
//@                    (\forall int i; 0<=i && i<list.size();
//@                       list.get(i)!=null));
```

TdE 16/07/2019

- Si consideri la seguente specifica del Binary Tree, che modella un albero binario di elementi non nulli di tipo T

```
public class BinaryTree<T> {
// Costruisce un albero binario con un solo elemento nella sua radice.
public BinaryTree(T rootElem) ;
// Ritorna l'elemento alla radice dell'albero
public /*@ pure @*/ T getRoot() ;
// Ritorna un Iterator che itera esattamente una volta su ogni elemento nell'albero.
public /*@ pure @*/ Iterator<T> allElements() ;

// Inserisce l'elemento elem come figlio sinistro del nodo che contiene l'element  father, purchè
tale figlio sinistro non esista già, in tal caso solleva l'eccezione NonEmptyChildException. Se father
non è parte dell'albero, solleva l'eccezione NoSuchElementException. Se l'elemento elem è già
parte dell'albero solleva l'eccezione ExistingElementException.
public void insertLeft(T father, T elem) throws NonEmptyChildException,
NoSuchElementException, ExistingElementException ;

// Inserisce l'elemento elem come figlio destro del nodo che contiene l'element father, purchè tale
figlio destro non esista già, in tal caso solleva l'eccezione NonEmptyChildException. Se father non
`e parte dell'albero, solleva l'eccezione NoSuchElementException. Se l'elemento elem `e già parte
dell'albero solleva l'eccezione ExistingElementException.
public void insertRight(T father, T elem) throws NonEmptyChildException,
NoSuchElementException, ExistingElementException ;
```

```
// Ritorna true se l'elemento elem `e parte dell'albero, falso altrimenti.
public /*@ pure @*/ boolean contains(T elem) ;
// Ritorna l'elemento a sinistra dell'elemento father (null se tale figlio sinistro non es
Solleva l'eccezione NoSuchElementException se father non `e parte dell'albero.
public /*@ pure @*/ T left(T father) throws NoSuchElementException ;
// Ritorna l'elemento a destra dell'elemento father (null se tale figlio destro non esist
Solleva l'eccezione NoSuchElementException se father non `e parte dell'albero.
public /*@ pure @*/ T right(T father) throws NoSuchElementException ;
// Ritorna l'elemento padre dell'elemento elem; null se elem `e la radice dell'albero.
l'eccezione NoSuchElementException se elem non `e parte dell'albero.
public /*@ pure @*/ T father(T elem) throws NoSuchElementException ;
}
```

TdE 16/07/2019

- Si consideri la seguente specifica del Binary Tree, che modella un albero binario di elementi non nulli di tipo T
- Si fornisca l’invariante pubblico per la classe Binary Tree

TdE 16/07/2019 – soluzione

- Partiamo ancora una volta da ciò che «deve essere vero».

1. Dato un albero, getRoot() non deve restituire null, il valore di getRoot() deve far parte dell'albero e non devono esserci valori nulli

TdE 16/07/2019 – soluzione

- Partiamo ancora una volta da ciò che «deve essere vero».

1. Dato un albero, getRoot() non deve restituire null, il valore di getRoot() deve far parte dell'albero e non devono esserci valori nulli

- //@public invariant

getRoot()!=null && contains(getRoot()) && !contains(null) &&

TdE 16/07/2019 – soluzione

- Partiamo ancora una volta da ciò che «deve essere vero».

2. Dato un elemento **e** nella struttura, se **e** è contenuto, allora lo sono il suo figlio destro e sinistro.

TdE 16/07/2019 – soluzione

- Partiamo ancora una volta da ciò che «deve essere vero».

2. Dato un elemento **e** nella struttura, se **e** è contenuto, allora lo sono il suo figlio destro e sinistro.

- (\forall T el; contains(el) && left(el)!=null; contains(left(el))) &&
- (\forall T el; contains(el) && right(el)!=null; contains(right(el)))

TdE 16/07/2019 – soluzione

- Partiamo ancora una volta da ciò che «deve essere vero».

3. Se un nodo è radice, il padre è **null.** Altrimenti, il padre è contenuto nell’albero

TdE 16/07/2019 – soluzione

- Partiamo ancora una volta da ciò che «deve essere vero».

3. Se un nodo è radice, il padre è **null.** Altrimenti, il padre è contenuto nell'albero

- (\forall T el; contains(el); getRoot().equals(el) ?
  father(el) == null : father(el) != null && contains(father(el))) &&

TdE 16/07/2019 – soluzione

- Ora scendiamo un po’ più nel dettaglio.

4. Dato un elemento **el,** se **el** ha un figlio sinistro, allora il padre di quel figlio sinistro è proprio **el**. Il discorso è analogo per il figlio destro

TdE 16/07/2019 – soluzione

- Ora scendiamo un po’ più nel dettaglio.

4. Dato un elemento **el,** se **el** ha un figlio sinistro, allora il padre di quel figlio sinistro è proprio **el**. Il discorso è analogo per il figlio destro

- (\forall T el; contains(el) && left(el)!=null ; father(left(el)).equals(el)) &&
  (\forall T el; contains(el) && right(el)!=null ; father(right(el)).equals(el)) &&

TdE 16/07/2019 – soluzione

- Ancora più in dettaglio.

5. Un nodo, figlio di un altro nodo, sarà sicuramente diverso dalla radice. Cosi mi sto anche assicurando che i nodi siano diversi tra loro.

TdE 16/07/2019 – soluzione

- Ancora più in dettaglio.

5. Un nodo, figlio di un altro nodo, sarà sicuramente diverso dalla radice. Cosi mi sto anche assicurando che i nodi siano diversi tra loro.

- (\forall T el; contains(el);

!left(el).equals(getRoot()) && !right(el).equals(getRoot() &&

TdE 16/07/2019 – soluzione

- Ancora più in dettaglio.

5. Un nodo, figlio di un altro nodo, sarà sicuramente diverso dalla radice. Cosi mi sto anche assicurando che i nodi siano diversi tra loro.

TdE 16/07/2019 – soluzione

- Ancora più in dettaglio.

5. Un nodo, figlio di un altro nodo, sarà sicuramente diverso dalla radice. Cosi mi sto anche assicurando che i nodi siano diversi tra loro.

- (\forall T el; contains(el);

!left(el).equals(getRoot()) && !right(el).equals(getRoot() &&

TdE 16/07/2019 – soluzione

- Ancora più in dettaglio.

6. Dati due elementi e1 ed e2 distinti, ci sono delle «condizioni» sui loro figli. In altre parole, un figlio sinistro di e1. Ovvero:
- i figli sinistri di e1 sono diversi dai figli sinistri di e2
- i figli destri di e1 sono diversi dai figli destri di e2
- i figli sinistri di e1 sono diversi dai figli destri di e2
- i figli destri di e1 sono diversi dai figli sinistri di e2

TdE 16/07/2019 – soluzione

- Ancora più in dettaglio.

6. Dati due elementi e1 ed e2 distinti, ci sono delle «condizioni» sui loro figli. In altre parole, un figlio sinistro di e1. Ovvero:
- i figli sinistri di e1 sono diversi dai figli sinistri di e2
- i figli destri di e1 sono diversi dai figli destri di e2
- i figli sinistri di e1 sono diversi dai figli destri di e2
- i figli destri di e1 sono diversi dai figli sinistri di e2

(\forall T el1; contains(el1); (\forall T el2; contains(el2) && !el1.equals(el2);

!left(el1).equals(left(el2)) && !left(el1).equals(right(el2)) &&

- //@ !right(el1).equals(left(el2)) && !right(el1).equals(right(el2))))

TdE 16/07/2019 – soluzione

- Ancora più in dettaglio.

6. Dati due elementi e1 ed e2 distinti, ci sono delle «condizioni» sui loro figli. In altre parole, un figlio sinistro di e1. Ovvero:
- i figli sinistri di e1 sono diversi dai figli sinistri di e2
- i figli destri di e1 sono diversi dai figli destri di e2
- i figli sinistri di e1 sono diversi dai figli destri di e2
- i figli destri di e1 sono diversi dai figli sinistri di e2

(\forall T el1; contains(el1); (\forall T el2; contains(el2) && !el1.equals(el2);

!left(el1).equals(left(el2)) && !left(el1).equals(right(el2)) &&

- //@ !right(el1).equals(left(el2)) && !right(el1).equals(right(el2))))

TdE 14/01/2023

- Si consideri la seguente classe SpeedLimits per memorizzare i limiti di velocità per i vari segmenti (RoadSegments) di una strada. **La strada inizia al km 0** e ha una lunghezza di **len() km**. Il costruttore assegna lo stesso limite di velocità all'intera strada. **Un segmento** (RoadSegment) **è** definito come un **tratto** di strada che **inizia** e **termina con** un **cambio di limite di velocità e all'interno** del quale il **limite** di velocità **non cambia**. Il primo segmento inizia al km 0 e l'ultimo segmento termina al km len() (escluso).

```
public class SpeedLimits {
// Considera una strada di lunghezza len e imposta a speedLimit il limite di velocita' per l'intera strada.
public SpeedLimits(int speedLimit, int len);
// Ritorna la lunghezza della strada in km.
public /*@ pure @*/ int len();
// Ritorna il limite di velocita' al km specificato.
public /*@ pure @*/ int getSpeedLimitAt(int km);
// Imposta il limite di velocita' a speedLimit per il tratto di strada che inizia al km start (incluso) e finisce al km end (escluso). Il limite di velocita' deve essere positivo e inferiore o uguale a 130 km orari. Lancia una InvalidBoundariesException se gli estremi del tratto di strada non sono validi.
public void changeSpeedLimit(int start, int end, int speedLimit) throws InvalidBoundariesException;
// Ritorna la lista dei segmenti contenuti nella strada. I segmenti sono ritornati in ordine (da quello che inizia per primo a quello che inizia per ultimo).
public /*@ pure @*/ List<RoadSegment> getSegments();
}
```

```
public /*@ pure @*/ RoadSegment {
// Ritorna il km di inizio del segmento (incluso nel segmento).
public /*@ pure @*/ int getStart();
// Ritorna il km di fine del segmento (escluso dal segmento).
public /*@ pure @*/ int getEnd();
// Ritorna il limite di velocita' definito per il segmento di strada
```

TdE 14/01/2023

Domanda c)

Si consideri la seguente implementazione della classe `SpeedLimits` che usa una mappa per definire i limiti di velocità e un intero per definire la lunghezza della strada. La mappa ha come chiave il km al quale inizia un segmento di strada e come valore il limite di velocità definito per quel segmento di strada.

```
public class SpeedLimits {
  private Map<Integer, Integer> segments;
  private int len;
... }
```

Per tale implementazione, si forniscano l'invariante di rappresentazione e la funzione di astrazione.

TdE 14/01/2023

- **Representation Invariant**
  Osservando gli attributi, devo dire tutto ciò che è **sempre vero** sugli attributi.

1. La len deve essere sempre positiva.

\\@private Invariant

len > 0

TdE 14/01/2023

- **Representation Invariant**
  Osservando gli attributi, devo dire tutto ciò che è **sempre vero** sugli attributi.

2. La mappa non deve essere nulla

\\@private Invariant

len > 0 && segments != null

TdE 14/01/2023

- **Representation Invariant**
  Osservando gli attributi, devo dire tutto ciò che è **sempre vero** sugli attributi.

3. La mappa deve contenere il km 0, inizio della strada

\\@private Invariant

len > 0 && segments != null && segments.containsKey(0)

TdE 14/01/2023

- **Representation Invariant**
  Osservando gli attributi, devo dire tutto ciò che è **sempre vero** sugli attributi.

3. La mappa è di **integer**, quindi devo controllare che gli elementi siano diversi da null. Inoltre, la velocità è sempre al più 130

(\forall Integer k; segments.containsKey(k); k != null && k >= 0 && k < len && segments.get(k) != null && segments.get(k) >= 0 && segments.get(k) <= 130) &&

TdE 14/01/2023

- **Representation Invariant**
  Osservando gli attributi, devo dire tutto ciò che è **sempre vero** sugli attributi.

3. Dato che non è possibile che due segmenti consecutivi abbiano lo stesso limite (vedi traccia), allora devo controllare che, se due segmenti hanno lo stesso limite, non sono consecutivi

In altre parole, se due segmenti A e B, con B > A, hanno lo stesso limite di velocità, allora deve esisterne un terzo, con limite di velocità diverso, compreso tra i due

TdE 14/01/2023

- **Representation Invariant**
  Osservando gli attributi, devo dire tutto ciò che è **sempre vero** sugli attributi.

3. Dato che non è possibile che due segmenti consecutivi abbiano lo stesso limite (vedi traccia), allora devo controllare che, se due segmenti hanno lo stesso limite, non sono consecutivi

```
(\forall Integer i, segments.keySet().containsKey(i);
        (\forall Integer j; j > i && segments.keySet().containsKey(j) ;
                \segments.get(i).equals(segments.get(j)) =>
                        (\exist Integer k, segments.keySet().containsKey(k) &&
                                i<=k<=J, !segments.get(k).equals(segments.get(i))
                        )
        )
)
```

TdE 14/01/2023

- **Funzione di Astrazione**
  Osservando i metodi pure utilizzati per descrivere il JML, in questo caso, len() e getSpeedLimitAt(), la funzione di astrazione deve permettermi di descrivere l'output.

1. Per len() posso utilizzare il parametro che ho già per la lunghezza della strada.

\\ private invariant

len() == len;

TdE 14/01/2023

- **Funzione di Astrazione**
  Osservando i metodi pure utilizzati per descrivere il JML, in questo caso, len() e getSpeedLimitAt(), la funzione di astrazione deve permettermi di descrivere l'output.

2. Per l'altra funzione, dato un valore in input, devo prendere il limite di velocità della chiave più alta, tra quelle più piccole dell'input

```
\\ private invariant
len() == len;
(\forall int i; i >= 0 && i < len;
        (\max Integer j; segments.containsKey(j) && j<=i; getSpeedLimitAt(i)
        == segments.get(j) ) )
```